

Подготовка к экзамену по C++ (ИТМО)

Тема 1: Преобразования типов (Явные и неявные)

Hardcore Theory Edition

14 июня 2026 г.

1 Неявные преобразования (Implicit Conversions)

Компилятор часто пытается "помочь" программисту и сам конвертирует типы там, где они не совпадают. Это источник огромного числа багов.

1.1 Опасные ситуации

- **Сужение (Narrowing):** Присвоение `double` в `int` обрежет дробную часть. Присвоение `int` в `char` обрежет старшие байты.
- **Signed vs Unsigned:** При сравнении `int` и `unsigned int`, знаковое число неявно приводится к беззнаковому! Если `int x = -1`, а `unsigned int y = 1`, то выражение `x > y` вернет `true`, так как `-1` превратится в гигантское положительное число (`0xFFFFFFFF`).
- **Конструкторы:** Если у класса есть конструктор от одного аргумента (например, `String(int size)`), компилятор может неявно превратить число в строку в местах, где ожидается объект.

Суть на пальцах (Жизненная аналогия)

Неявное преобразование — это попытка перелить воду из 5-литрового ведра (`long`) в 1-литровую банку (`short`). Компилятор C++ сделает это молча. Вода польется через край (потеря данных), но программа продолжит работать, как ни в чем не бывало.

2 Ключевое слово `explicit`

Специально придумано, чтобы запретить компилятору заниматься "самодеятельностью" с вашими классами. Если вы пишете `explicit MyClass(int x)`, компилятор запретит код вида `MyClass obj = 5;` или передачу числа в функцию, ожидающую `MyClass`. Вам придется писать явно: `MyClass obj(5);`. То же самое касается операторов приведения: `explicit operator bool()` запретит складывать ваш объект с числами, разрешив использовать его только внутри `if(obj)`.

3 Явные преобразования: 4 всадника C++ Casts

В языке Си можно было написать `(int*)ptr`. Этот C-style cast ужасен: он пытается применить разные методы каста по очереди (игнорируя `const`, меняя иерархию), пока не получится. Найти его поиском по коду невозможно. В C++ ввели 4 строгих оператора.

3.1 1. `static_cast` (Рабочая лошадка)

Когда использовать: Для предсказуемых, легальных конверсий на этапе компиляции.

- Преобразования чисел (`float` в `int`).
- Приведение `void*` к конкретному указателю.
- Перемещение по иерархии классов (Upcast и Downcast), НО компилятор не проверяет, действительно ли базовый класс является нужным наследником! Если опшибетесь — UB.

3.2 2. `dynamic_cast` (Безопасность в ООП)

Когда использовать: Для безопасного **Downcast** (от базового к наследнику) в иерархии с виртуальными функциями (полиморфизм).

- Работает на этапе выполнения (Runtime). Использует RTTI (Run-Time Type Information).
- Если вы попытаетесь скастовать `Base*` в `Derived*`, а под капотом лежит вовсе не `Derived`, `dynamic_cast` не уронит программу, а аккуратно вернет `nullptr`.
- Работает медленнее других кастов.

3.3 3. `const_cast` (Нарушитель правил)

Когда использовать: Исключительно для снятия (или добавления) квалификаторов `const` и `volatile`.

- **Важно:** Если изначальная переменная была объявлена как константа (например, `const int x = 5;`), а вы через `const_cast` попытаетесь её изменить — это UB.
- Легально использовать, когда вы точно знаете, что изначальный объект не константный, но он дошел до вас в виде константной ссылки или указателя (часто нужно при работе с древними C-библиотеками).

3.4 4. `reinterpret_cast` (Жесткий взлом)

Когда использовать: Почти никогда. Только для системного программирования.

- Говорит компилятору: "Забудь про типы, просто возьми биты по этому адресу и считай их другим типом".
- Позволяет скастовать указатель в целое число (и обратно) или указатель одного типа в совершенно не связанный указатель другого типа.

Суть на пальцах (Жизненная аналогия)

Представьте себе проходную в корпорации:

1. `static_cast`: Перевод из отдела в отдел. HR проверяет бумажку (компиляцию), и если ты бухгалтер, тебя пустят в отдел финансов.
2. `dynamic_cast`: Охранник с рацией (Runtime). Если ты с бейджиком "Сотрудник" попытаешься зайти в "Кабинет Директора" охранник проверит тебя по базе. Если ты не директор — мягко развернет (`nullptr`).
3. `const_cast`: Ты заклеил слово "Посетитель" на бейджике изолентой, чтобы пройти.
4. `reinterpret_cast`: Ты пробил стену кувалдой и зашел куда захотел. Здание может обрушиться.